Parcial práctico — Sistema de inventario y ventas

Asignatura: Acceso a Datos Temas: Python, SQLite, CRUD, relaciones y transacciones
Modalidad: Práctica individual Valor: 100 puntos

Hecho: Santiago Enrique Noriega Garcia

1. Contexto del problema

Una tienda de componentes electrónicos necesita un programa en Python para administrar sus productos y registrar ventas.

Actualmente, la tienda tiene dificultades para controlar las existencias. En ocasiones registra una venta, pero no actualiza el inventario, generando inconsistencias.

Tu misión será desarrollar una aplicación de consola que almacene los datos en SQLite y garantice que las ventas se registren correctamente.

2. Actividades

Parte 1. Análisis y diseño (20 puntos)

Antes de programar:

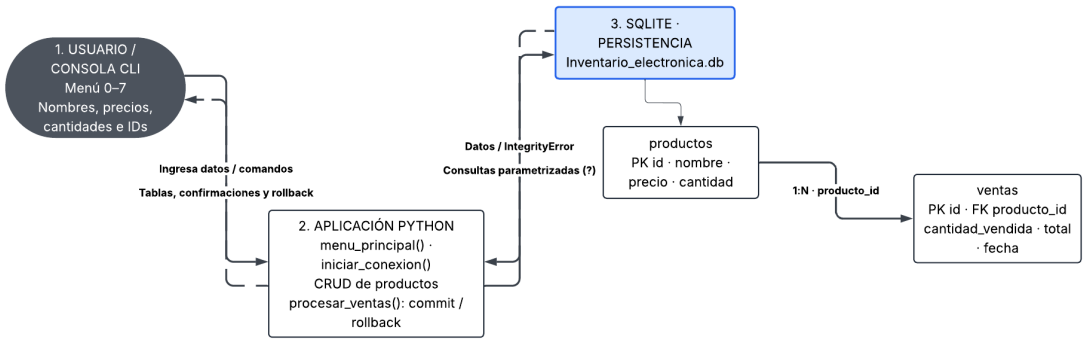
1. Identifica las entidades y atributos necesarios.

De acuerdo con el esquema definido en inicializar_bd(), la base de datos Inventario_electronica.db cuenta con dos entidades relacionadas mediante clave foránea:

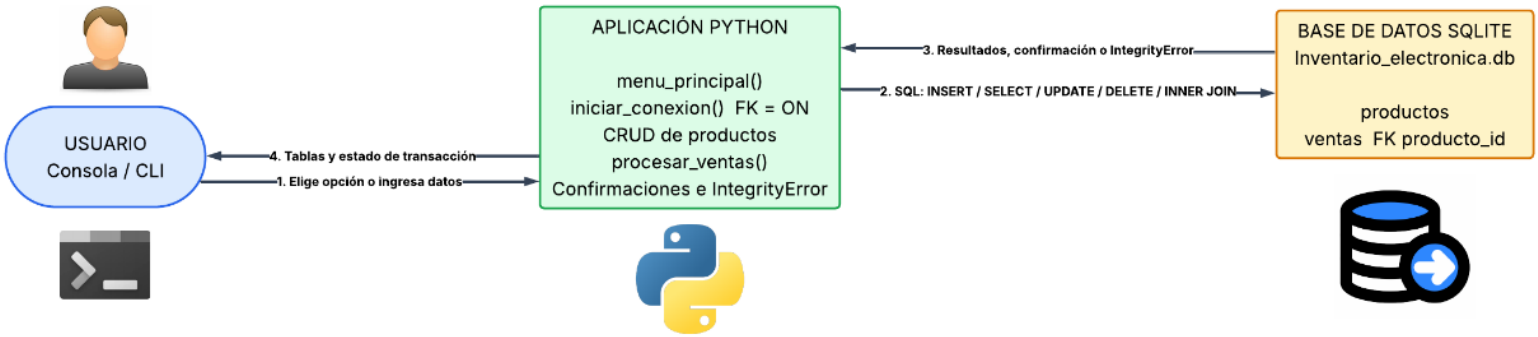
- **Tabla productos**:
    - id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único de cada producto.
    - nombre (TEXT NOT NULL): Nombre del componente electrónico.
    - precio (REAL NOT NULL CHECK(precio > 0)): Precio unitario validado para ser mayor que cero.
    - cantidad (INTEGER NOT NULL CHECK(cantidad >= 0)): Cantidad disponible en inventario (no negativa).
- **Tabla ventas**:
    - id (INTEGER PRIMARY KEY AUTOINCREMENT): Identificador único de cada registro de venta.

- producto_id (INTEGER NOT NULL): Clave foránea (FOREIGN KEY) que referencia a productos(id).
- cantidad_vendida (INTEGER NOT NULL CHECK(cantidad_vendida > 0)): Unidades vendidas en la operación.
- total (REAL NOT NULL CHECK(total > 0)): Valor total pagado (precio_unitario * cantidad_vendida).
- fecha (TIMESTAMP DEFAULT CURRENT_TIMESTAMP): Marca temporal de la venta generada desde Python con datetime.now().strftime("%Y-%m-%d %H:%M:%S") o por defecto en SQLite.

2. Diseña una base de datos con al menos dos tablas relacionadas mediante una clave foránea.



3. Dibuja un diagrama sencillo que represente cómo interactúan el usuario, Python y SQLite.



4. Explica qué operaciones deben ejecutarse al registrar una venta.

- **Validación inicial de entrada**: En registrar_venta_menu() y procesar_ventas() se comprueba que el ID y la cantidad_vendida sean enteros mayores a 0.
- **Verificación de existencia del producto**: Se abre la conexión con iniciar_conexion() (que activa PRAGMA foreign_keys = ON) y se ejecuta SELECT id, nombre, precio, cantidad FROM productos WHERE id = ?.

- **Comprobación de stock disponible**: Se evalúa si stock_actual < cantidad_vendida. Si no hay unidades suficientes, se detiene el proceso e informa al usuario.

- **Inserción del registro de venta**: Se calcula total_venta y fecha_actual, y se ejecuta INSERT INTO ventas (producto_id, cantidad_vendida, total, fecha) VALUES (?, ?, ?, ?).

- **Simulación de error opcional (Reto obligatorio)**: Si simular_error=True, se lanza un RuntimeError controlado justo después del INSERT y antes del UPDATE.

- **Actualización del inventario**: Si no hay errores, se descuentan las unidades mediante UPDATE productos SET cantidad = cantidad - ? WHERE id = ?.

- **Confirmación (commit()) o Reversión (rollback())**: Si ambos pasos se completan, se llama a conn.commit(). Si ocurre el RuntimeError o cualquier fallo de base de datos, el bloque except Exception as e: ejecuta conn.rollback(), revirtiendo el INSERT en ventas y manteniendo el stock intacto.

Parte 2. Desarrollo del programa (50 puntos)

Implementa una aplicación Python que utilice `sqlite3` y permita:

1. Registrar productos (nombre, precio y cantidad disponible).

```
================================================================
   SISTEMA DE INVENTARIO Y VENTAS - COMPONENTES ELECTRÓNICOS
================================================================
1. Registrar producto
2. Consultar todos los productos
3. Actualizar producto
4. Eliminar producto
5. Registrar una venta
6. Mostrar historial de ventas
0. Salir
================================================================
Seleccione una opción (0-6): 1

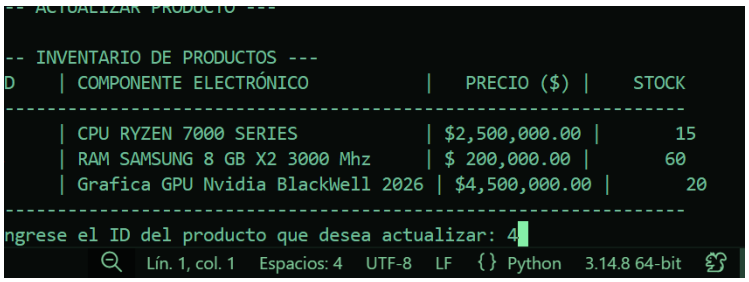
--- REGISTRAR NUEVO PRODUCTO ---
Nombre del componente electrónico: CPU RYZEN 7000 SERIES
Precio unitario ($): 2500000
Cantidad inicial en inventario: 15
[OK] Producto 'CPU RYZEN 7000 SERIES' registrado con éxito (ID asignado: 3).
```

2. Consultar todos los productos.

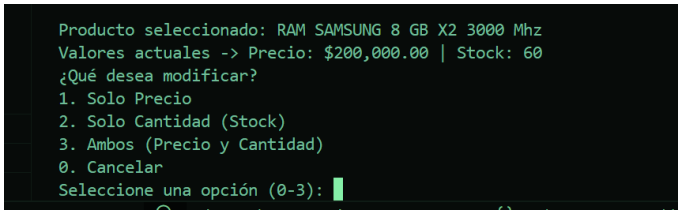
```
-- INVENTARIO DE PRODUCTOS ---
D    | COMPONENTE ELECTRÓNICO           |   PRECIO ($) |    STOCK
-------------------------------------------------------------------
     | CPU RYZEN 7000 SERIES            | $2,500,000.00 |       15
     | RAM SAMSUNG 8 GB X2 3000 Mhz     | $ 200,000.00 |       60
     | Grafica GPU Nvidia BlackWell 2026 | $4,500,000.00 |       20
-------------------------------------------------------------------
```



3. Actualizar precio o cantidad de un producto.



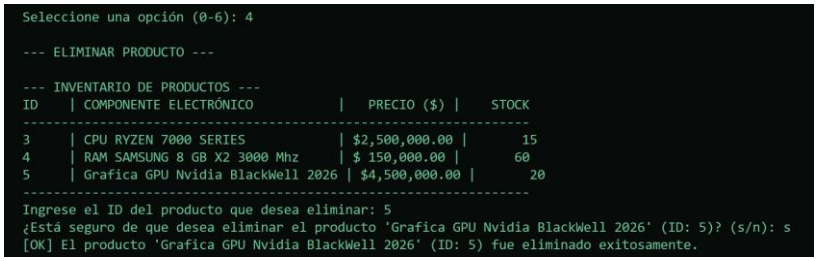
El id de la memoria ram que seria 4 el precio lo bajaremos en 150000



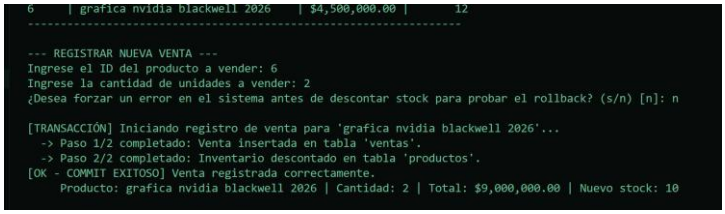
Nos preguntara si queremos cambiar el precio o el stock en este caso será el precio



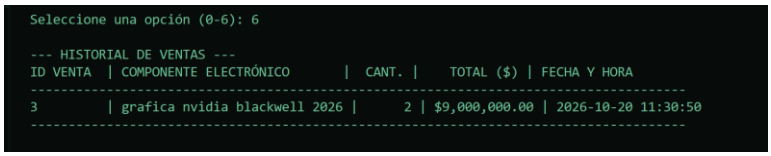
4. Eliminar productos, respetando las relaciones de la base de datos.

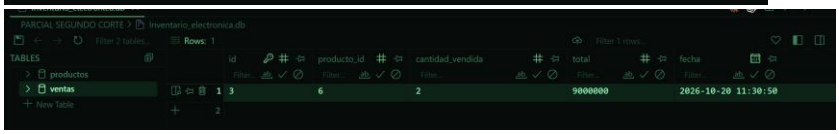


5. Registrar una venta indicando el producto y la cantidad.



6. Mostrar un historial de ventas con el nombre del producto mediante `JOIN`.





La aplicación debe contar con un menú interactivo, crear automáticamente las tablas necesarias y utilizar consultas SQL parametrizadas (`?`).

Parte 3. Transacciones y validaciones (30 puntos)

Cuando se realiza una venta, el programa debe:

1. Verificar que el producto exista.

```
eleccione una opcion (0-6): 5

-- INVENTARIO DE PRODUCTOS ---
D    | COMPONENTE ELECTRÓNICO           |   PRECIO ($) |    STOCK
---------------------------------------------------------------------
     | CPU RYZEN 7000 SERIES            | $2,500,000.00 |       15
     | RAM SAMSUNG 8 GB X2 3000 Mhz     | $ 150,000.00 |       60
     | grafica nvidia blackwell 2026    | $4,500,000.00 |       10
---------------------------------------------------------------------

-- REGISTRAR NUEVA VENTA ---
ngrese el ID del producto a vender: 99
ngrese la cantidad de unidades a vender: 1
Desea forzar un error en el sistema antes de descontar stock para probar el rollback? (s/n) [n]: n
!] Error: No existe el producto con ID 99.
```

2. Comprobar que haya suficientes unidades.

```
Seleccione una opción (0-6): 5

--- INVENTARIO DE PRODUCTOS ---
ID    | COMPONENTE ELECTRÓNICO           |   PRECIO ($) |    STOCK
---------------------------------------------------------------------
3     | CPU RYZEN 7000 SERIES            | $2,500,000.00 |       15
4     | RAM SAMSUNG 8 GB X2 3000 Mhz     | $ 150,000.00 |       60
6     | grafica nvidia blackwell 2026    | $4,500,000.00 |       10
---------------------------------------------------------------------

--- REGISTRAR NUEVA VENTA ---
Ingrese el ID del producto a vender: 4
Ingrese la cantidad de unidades a vender: 70
¿Desea forzar un error en el sistema antes de descontar stock para probar el rollback? (s/n) [n]: n
[!] Advertencia: Stock insuficiente para 'RAM SAMSUNG 8 GB X2 3000 Mhz'. Disponibles: 60
Unidades solicitadas: 70
```

3. Registrar la venta en la base de datos.

```
--- INVENTARIO DE PRODUCTOS ---
ID    | COMPONENTE ELECTRÓNICO           |   PRECIO ($) |    STOCK
---------------------------------------------------------------------
3     | CPU RYZEN 7000 SERIES            | $2,500,000.00 |       15
4     | RAM SAMSUNG 8 GB X2 3000 Mhz     | $ 150,000.00 |       60
6     | grafica nvidia blackwell 2026    | $4,500,000.00 |       10
---------------------------------------------------------------------

--- REGISTRAR NUEVA VENTA ---
Ingrese el ID del producto a vender: 3
Ingrese la cantidad de unidades a vender: 10
¿Desea forzar un error en el sistema antes de descontar stock para probar el rollback? (s/n) [n]: s

=== GENERANDO ERROR CONTROLADO Y DEMOSTRANDO ROLLBACK ===

[TRANSACCIÓN] Iniciando registro de venta para 'CPU RYZEN 7000 SERIES'...
  -> Paso 1/2 completado: Venta insertada en tabla 'ventas'.
  -> [!] Provocando error controlado antes de descontar el inventario...

[ALERTA - EXCEPCIÓN DETECTADA]: Fallo simulado de conexión/sistema después de INSERT en 'ventas' y antes de UPDATE en 'productos'.
[ROLLBACK EJECUTADO] Se canceló toda la transacción.
La venta NO fue guardada y el stock del producto permanece intacto.

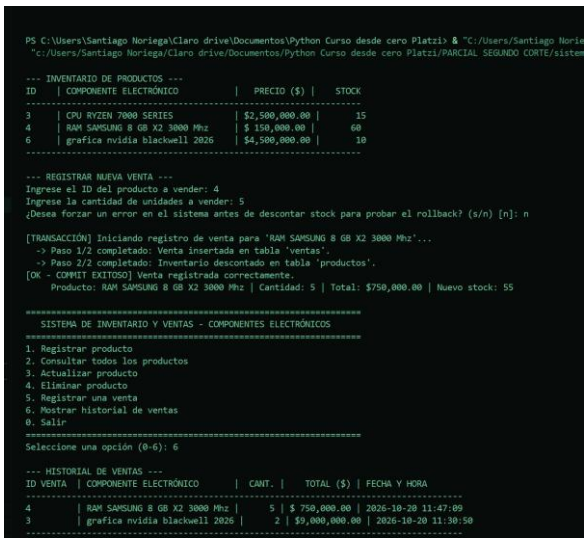
--- VERIFICACIÓN AUTOMÁTICA DE INTEGRIDAD TRAS EL ROLLBACK ---

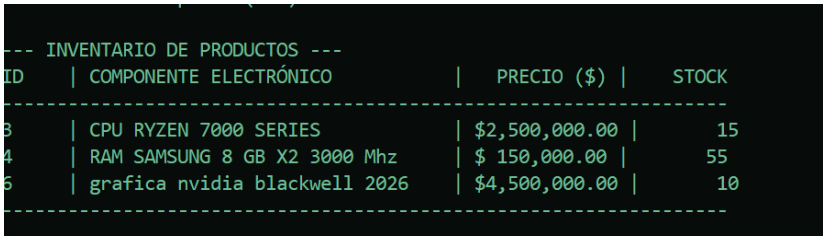
--- INVENTARIO DE PRODUCTOS ---
ID    | COMPONENTE ELECTRÓNICO           |   PRECIO ($) |    STOCK
---------------------------------------------------------------------
3     | CPU RYZEN 7000 SERIES            | $2,500,000.00 |       15
4     | RAM SAMSUNG 8 GB X2 3000 Mhz     | $ 150,000.00 |       60
6     | grafica nvidia blackwell 2026    | $4,500,000.00 |       10
---------------------------------------------------------------------

--- HISTORIAL DE VENTAS ---
ID VENTA  | COMPONENTE ELECTRÓNICO       |  CANT. |    TOTAL ($) | FECHA Y HORA
-------------------------------------------------------------------------------------
3         | grafica nvidia blackwell 2026 |      2 | $9,000,000.00 | 2026-10-20 11:30:50
-------------------------------------------------------------------------------------
```

Lo que ocurre es que cuando se provoca el error por otro lado inmediatamente cierra la operación después del insert en la tabla de eventos después captura la excepción que ejecuta el compromet back como se aparecía en la verificación de forma automática y junto lo que hace es que no desconta el stock que había si no queda nada más permanente pero la venta no se guarda porque literalmente generamos un fallo en la simulación

4. Descontar las unidades vendidas del inventario.
5. Confirmar ambas operaciones mediante `commit()`.





Reto obligatorio: si ocurre un error después de registrar la venta, pero antes de actualizar el inventario, el programa debe utilizar `rollback()` para cancelar toda la operación.

Demuestra este comportamiento provocando un error controlado y verificando que los datos permanezcan intactos.

3. Entrega

Sube el proyecto a GitHub y entrega el enlace del repositorio.

Debe contener el código Python, las instrucciones para ejecutarlo, un analisis de como esta construido el software (puedes usar diagramas para explicarlo), y evidencias de ejecución (pantallazos), el modelo de datos, el diagrama de arquitectura y una explicación breve de la transacción.